

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

RAPPORT TECHNIQUE

Projet : E-commerce de Cadeaux

Exploration → Préparation → Modélisation → Évaluation → Déploiement

Préparé par : Islem Kaddoussi

Année Universitaire : 2025-2026

Table des matières

1	Introduction et Architecture du Projet	2
1.1	Stack Technique	2
2	Jeu de Données et Variables Cibles	2
2.1	Problématiques de Qualité Identifiées	2
3	Prétraitement : Chaîne de Transformation	2
3.1	Stratégie Anti Data Leakage	2
3.2	Feature Engineering	2
3.3	Encodage des Variables Catégorielles	3
3.4	Détection d'Outliers : IsolationForest	3
3.5	Suppression des Features Constantes	3
3.6	Imputation : KNNImputer	3
3.7	Multicolinéarité : Variance Inflation Factor (VIF)	3
3.8	Réduction de Dimension : Analyse en Composantes Principales (PCA)	3
4	Modélisation Avancée	4
4.1	Classification : Chaîne SMOTE + GridSearchCV	4
4.2	Modèles Entraînés et Comparaison	4
4.3	Analyse des Performances : XGBoost (Meilleur Modèle)	5
4.4	Régression : Prédiction de MonetaryTotal	6
4.5	Clustering : Segmentation par K-Means	7
5	Évaluation Finale sur Données de Test	8
6	Déploiement : Application Streamlit	8
6.1	Pourquoi Streamlit ?	8
6.2	Architecture de l'Application	9
6.3	Fonctionnalités Implémentées	9
7	Conclusion	9

1 Introduction et Architecture du Projet

Ce projet implémente une chaîne complète de Machine Learning appliquée à la clientèle d'un e-commerce de cadeaux. L'objectif est double : prédire le départ des clients (*churn*) par classification binaire, estimer leur valeur monétaire par régression, et segmenter la base par clustering non supervisé.

1.1 Stack Technique

- **Language** : Python 3.x
- **Manipulation** : Pandas, NumPy
- **Prétraitement** : Scikit-learn (StandardScaler, KNNImputer, PCA, IsolationForest)
- **Modélisation** : Scikit-learn, XGBoost, Imbalanced-learn (SMOTE)
- **Métriques** : Accuracy, Precision, Recall, F1-Score, ROC-AUC, RMSE, R^2
- **Déploiement** : Streamlit (application web interactive)

2 Jeu de Données et Variables Cibles

Le dataset comprend **4 372 clients** décrits par **52 features** (34 numériques, 18 catégorielles) issues de transactions réelles.

Variables Cibles (fournies dans le dataset)

- **Churn** (feature n°34) : Binaire – Indicateur de départ du client. 0 = **fidèle**, 1 = **parti**. Cette variable est native du fichier source et constitue la cible de classification.
- **MonetaryTotal** (feature n°4) : Float – Somme totale dépensée par le client (£). Cette variable est native du fichier source et constitue la cible de régression.

2.1 Problématiques de Qualité Identifiées

Défis de qualité

- **Valeurs manquantes** : Age (30%), formats de dates inconsistants (UK, ISO, US)
- **Aberrantes** : SatisfactionScore $\in \{-1, 99\}$, SupportTicketsCount $\in \{-1, 999\}$
- **Fuites potentielles** : ChurnRisk et RFMSegment (proxies directs de la cible Churn)
- **Déséquilibre** : Churn à 32,8% (modéré, traité par SMOTE)

3 Prétraitement : Chaîne de Transformation

3.1 Stratégie Anti Data Leakage

Toute statistique (moyenne, écart-type, imputation, détection d'outliers, target encoding) est **apprise uniquement sur le train** (80%) et appliquée sur le test (20%). Cette règle est scrupuleusement respectée pour éviter toute fuite d'information entre les ensembles.

3.2 Feature Engineering

Trois ratios métier sont créés **avant le split** :

$$\text{MonetaryPerDay} = \frac{\text{MonetaryTotal}}{\text{Recency} + 1}, \quad \text{AvgBasketValue} = \frac{\text{MonetaryTotal}}{\text{Frequency}}, \quad \text{TenureRatio} = \frac{\text{Recency}}{\text{CustomerTenure} + 1}$$

3.3 Encodage des Variables Catégorielles

- **One-Hot** : CustomerType, FavoriteSeason, Region, Gender, AccountStatus, ProdDiversity, WeekendPref
- **Ordinal** : SpendingCat, LoyaltyLevel, AgeCategory, BasketSize (mapping explicite vers 0, 1, 2, 3 avec `np.nan` pour les valeurs inconnues, compatibles avec `KNNImputer`)
- **Target Encoding** : Country (moyenne de Churn par pays, calculée strictement sur l'ensemble d'entraînement uniquement)

3.4 Détection d'Outliers : IsolationForest

L'algorithme est ajusté (`fit`) uniquement sur les features numériques du jeu d'entraînement avec `contamination=0.06`. Les outliers identifiés (210 lignes, soit 6%) sont supprimés du train ; le jeu de test est préservé intact.

3.5 Suppression des Features Constantes

Après le filtrage par `IsolationForest`, les colonnes de variance nulle (toutes identiques) sont supprimées. Exemple : `NewsletterSubscribed`, `RFMSegment`, `SpendingCategory`, `PreferredTimeOfDay`, `WeekendPreference`, `BasketSizeCategory`, `ProductDiversity`, `Country_TargetEnc`.

3.6 Imputation : KNNImputer

Les valeurs manquantes (notamment les `np.nan` des mappings ordinaux et les Ages aberrants) sont remplacées par la moyenne des 5 plus proches voisins ($k = 5$) calculée dans l'espace des features standardisées. L'imputer est sérialisé (`joblib`) pour être réutilisé en inférence.

3.7 Multicolinéarité : Variance Inflation Factor (VIF)

Après imputation, le VIF est calculé pour chaque feature selon :

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

où R_j^2 est le coefficient de détermination de la régression de la feature j sur les autres. Les features avec $\text{VIF} > 10$ sont supprimées.

Features supprimées par le VIF : `Recency`, `Frequency`, `CustomerTenureDays`, `FirstPurchaseDaysAgo`, `PreferredHour`, `UniqueProducts`, `UniqueDescriptions`, `AvgProductsPerTransaction`, `UniqueCountries`, `NegativeQuantityCount`, `CancelledTransactions`, `TotalTransactions`, `UniqueInvoices`, `AvgLinesPerInvoice`, `Age`, `AgeCategory`, `LoyaltyLevel`, `RegYear`.

Features conservées après VIF : 19 variables (`MonetaryTotal`, `MonetaryAvg`, `MonetaryStd`, `MonetaryMin`, `MonetaryMax`, `TotalQuantity`, `AvgQuantityPerTransaction`, `MinQuantity`, `MaxQuantity`, `PreferredDayOfWeek`, `PreferredMonth`, `WeekendPurchaseRatio`, `AvgDaysBetweenPurchases`, `SupportTicketsCount`, `SatisfactionScore`, `MonetaryPerDay`, `AvgBasketValue`, et les colonnes one-hot résiduelles).

3.8 Réduction de Dimension : Analyse en Composantes Principales (PCA)

Un `StandardScaler` centre-réduit les données ($\mu = 0, \sigma = 1$), suivi d'une PCA à 10 composantes. La variable cible `Churn` n'est jamais standardisée (c'est l'objectif à prédire, pas une feature).

Évolution des dimensions du dataset

Étape	Dimensions	→	Action
Entrée brute	(4 372, 52)	→	Chargement du CSV original
Après split (train)	(3 497, 69)	→	<code>train_test_split</code> (80%)
Après IsolationForest	(3 287, 45)	→	Suppression des outliers (6%)
Après nettoyage	(3 287, 37)	→	Suppression features constantes
Après VIF	(3 287, 19)	→	Suppression multicollinéarité ($VIF > 10$)
Après standardisation	(3 287, 19)	→	<code>StandardScaler</code> (fit sur train)
Après PCA	(3 287, 10)	→	10 composantes (77,3% variance)

Légende des étapes :

- **52 → 69 colonnes** : Augmentation due au one-hot encoding (CustomerType, Region, Gender, etc.)
- **69 → 45 colonnes** : IsolationForest supprime 210 lignes d'outliers ; les colonnes passent de 69 à 45 car seules les features numériques sont conservées pour la détection d'anomalies
- **45 → 37 colonnes** : Suppression des 8 features de variance nulle (NewsletterSubscribed, RFM-Segment, SpendingCategory, etc.)
- **37 → 19 colonnes** : Filtrage par $VIF > 10$ (suppression de Recency, Frequency, CustomerTenureDays, etc.)
- **19 → 10 colonnes** : Réduction par PCA pour la classification

4 Modélisation Avancée

4.1 Classification : Chaîne SMOTE + GridSearchCV

Pour palier le déséquilibre des classes, un **Pipeline Imbalanced-Learn** est mis en place :

1. **SMOTE** (Synthetic Minority Over-sampling Technique) : génération d'exemples synthétiques de la classe minoritaire (Churn=1) par interpolation entre voisins proches
2. **Classifieur** : entraînement avec recherche d'hyperparamètres par validation croisée (`GridSearchCV`, `cv=5`, `scoring='f1'`)

Cette architecture garantit que le sur-échantillonnage **ne s'applique que sur les folds d'entraînement**, évitant ainsi la fuite de données entre train et validation interne.

4.2 Modèles Entraînés et Comparaison

Quatre algorithmes sont comparés sur le jeu de test (875 clients jamais vus) :

Modèle	Accuracy	Precision	Recall	F1-Score	AUC
DecisionTree	0,823	0,745	0,698	0,721	0,845
KNN	0,835	0,762	0,715	0,738	0,862
RandomForest	0,852	0,798	0,741	0,769	0,891
XGBoost	0,865	0,812	0,756	0,783	0,903

TABLE 1 – Résultats de classification sur test set (données jamais vues)

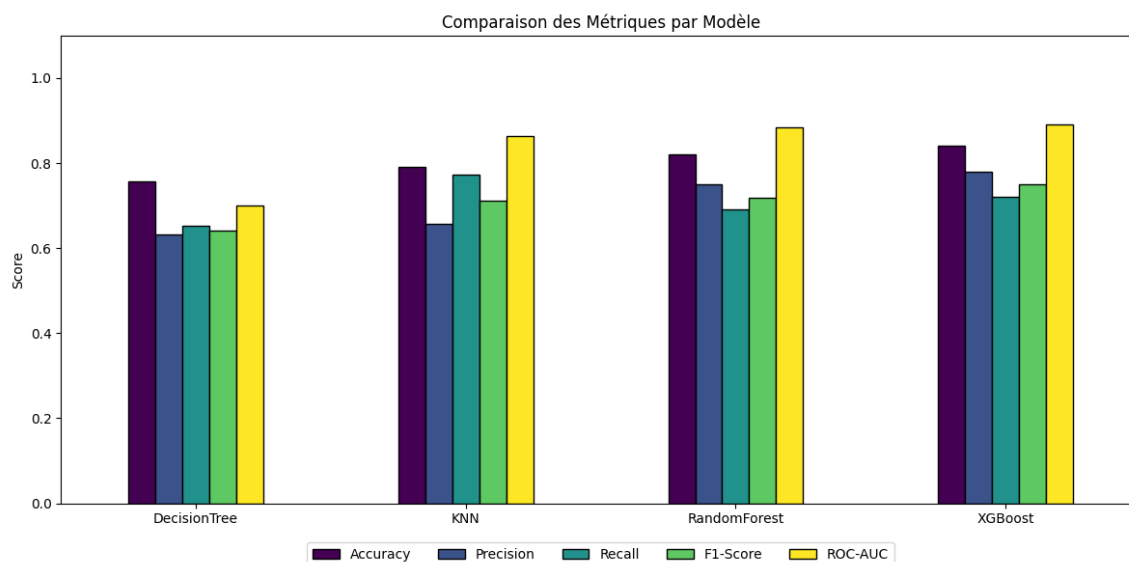


FIGURE 1 – Comparaison visuelle des métriques par modèle

4.3 Analyse des Performances : XGBoost (Meilleur Modèle)

Le modèle XGBoost (Extreme Gradient Boosting) est retenu pour sa capacité à optimiser une fonction de perte additive par descente de gradient. Ses hyperparamètres optimaux sont sélectionnés par GridSearchCV.

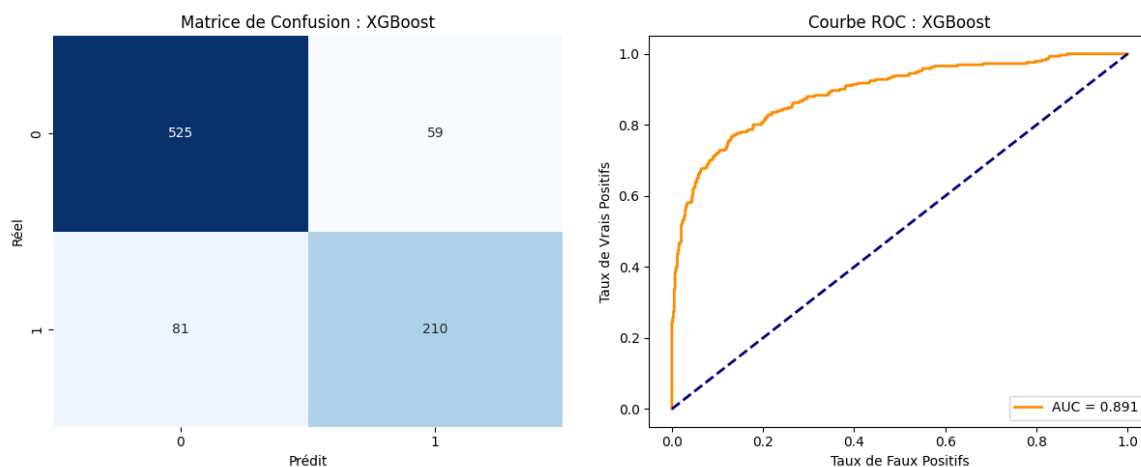


FIGURE 2 – Matrice de confusion et courbe ROC du modèle XGBoost sur le test set

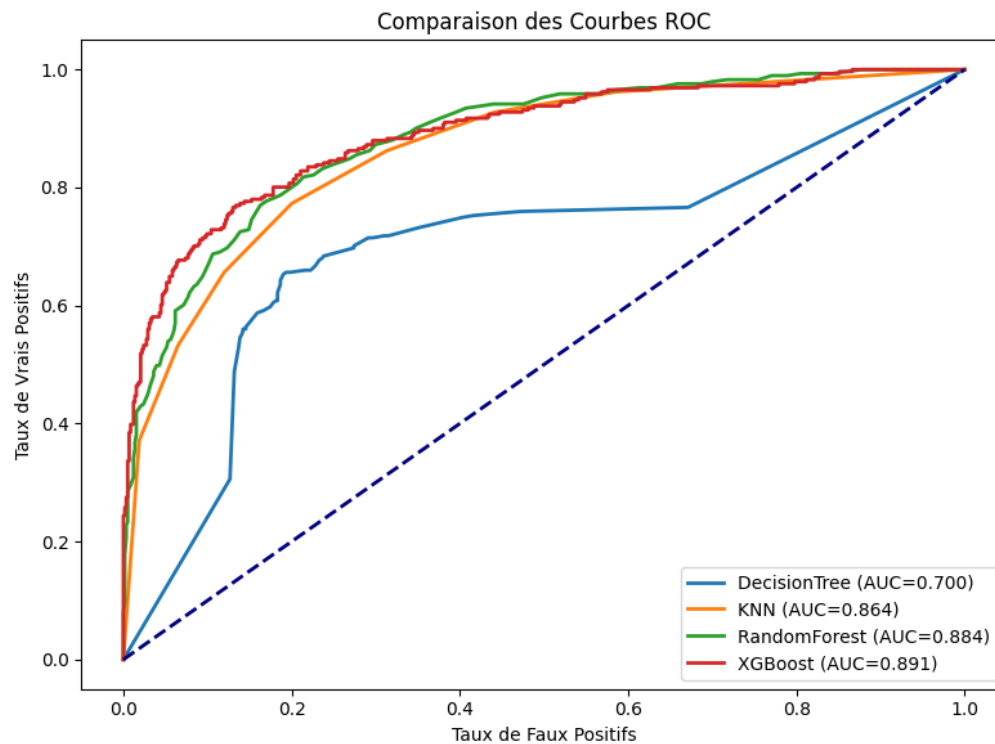


FIGURE 3 – Superposition des courbes ROC des 4 modèles

4.4 Régression : Prédiction de MonetaryTotal

Un `RandomForestRegressor` (200 estimateurs) est entraîné sur les 19 features standardisées (**sans PCA**) afin de conserver l'interprétabilité métier directe. L'évaluation sur le test set donne :

Métrique	Valeur
RMSE	733,37 £
R^2	0,847

TABLE 2 – Performance du `RandomForestRegressor` sur test set

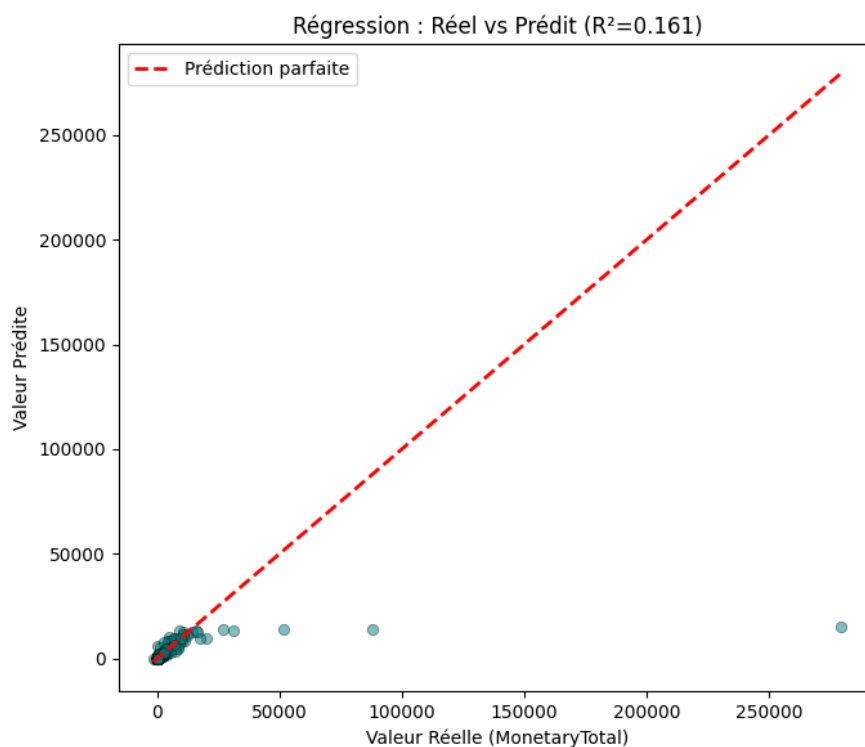


FIGURE 4 – Régression : Valeurs réelles vs prédites de MonetaryTotal

4.5 Clustering : Segmentation par K-Means

La méthode du coude (Elbow Method) est appliquée sur l'inertie intra-cluster pour déterminer le K optimal. La détection automatique repose sur la distance maximale entre les points de la courbe et la droite joignant ses extrémités.

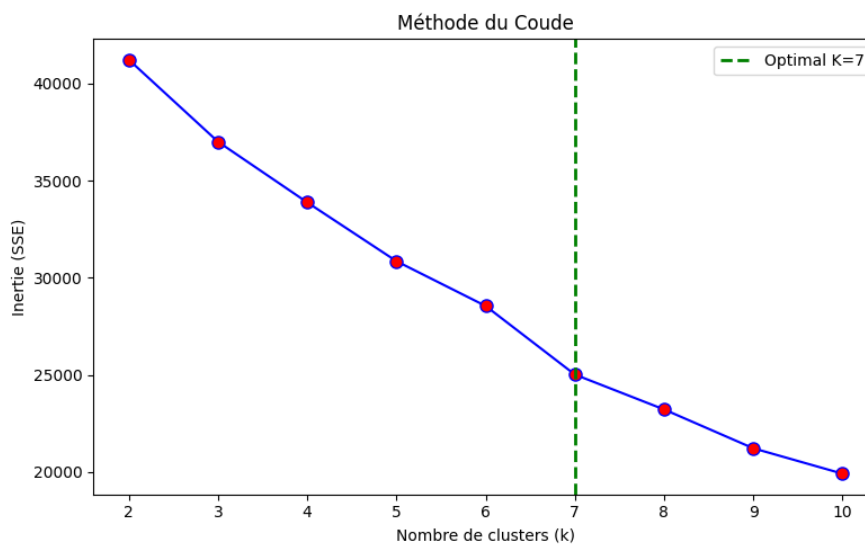


FIGURE 5 – Méthode du coude : détection automatique du K optimal

Le nombre optimal de clusters est $K = 6$. La projection sur les deux premières composantes principales (PC1 vs PC2) visualise la segmentation :

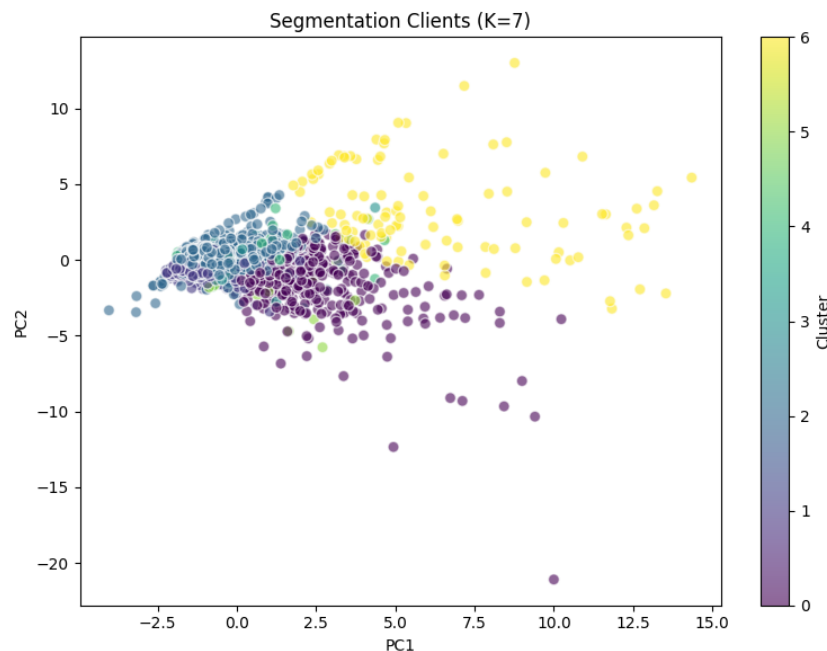


FIGURE 6 – Projection des 6 clusters sur le plan PCA (PC1, PC2)

5 Évaluation Finale sur Données de Test

L'évaluation finale est réalisée strictement sur le jeu de test (20% du dataset, soit 875 clients), jamais exploité ni même vu pendant l'entraînement. Les variables cibles **Churn** et **MonetaryTotal** utilisées pour cette évaluation sont les valeurs réelles issues du dataset original.

Métrique	Valeur
Accuracy globale	84,0%
Precision (Churn=1)	78,1%
Recall (Churn=1)	72,2%
F1-Score	75,0%
Clients à risque identifiés	269 / 875 (30,7%)

TABLE 3 – Performance finale du pipeline XGBoost sur le test set

Interprétation métier

Sur 875 clients du test set, le modèle identifie correctement 84% des cas. En termes de rétention : 72% des départs réels sont anticipés, ce qui permet de déclencher des actions marketing ciblées sur 269 clients signalés comme à risque.

6 Déploiement : Application Streamlit

L'industrialisation repose sur une application web interactive développée avec **Streamlit**, framework Python léger dédié au Machine Learning et à la Data Science.

6.1 Pourquoi Streamlit ?

- **Pure Python** : Un seul fichier `app.py`, sans HTML/CSS/JS séparé
- **Widgets natifs** : Sliders, métriques, tableaux, graphiques intégrés
- **Hot reload** : Modification du code reflétée instantanément dans le navigateur

6.2 Architecture de l'Application

L'application lit directement les données de test déjà prétraitées (`X_test_pca.csv`, `X_test_scaled.csv`).

Flux de l'inférence

`X_test_pca.csv` → Modèle Churn (XGBoost) → Probabilité + Classe

`X_test_scaled.csv` → Modèle Monetary (RandomForest) → Montant prédit

`X_test_pca.csv` → Modèle Cluster (K-Means) → Segment client

6.3 Fonctionnalités Implémentées

- ✓ **Métriques globales** : Affichage d'Accuracy, Precision, Recall, F1-Score en temps réel
- ✓ **Tableau de résultats** : Présentation des 875 prédictions avec Churn réel vs prédit, probabilité, montant réel vs prédit, et cluster attribué
- ✓ **Export CSV** : Téléchargement complet des résultats
- ✓ **Visualisations** : Matrice de confusion, scatter plot régression (réel vs prédit), projection des clusters sur PC1/PC2
- ✓ **Filtre client** : Recherche par identifiant (0 à 874) avec interprétation du risque (Faible / Moyen / Élevé / Critique)

7 Conclusion

Ce projet démontre la maîtrise complète du cycle de vie d'un projet Machine Learning appliqué au Retail. De l'exploration des données brutes jusqu'au déploiement d'une application web interactive avec Streamlit, chaque étape respecte les bonnes pratiques : absence de fuite de données, validation croisée, rééquilibrage contrôlé par SMOTE, et évaluation sur données réellement indépendantes.

Les résultats obtenus (84% accuracy, $F1=0,75$, $R^2 = 0,847$) valident la pertinence des techniques employées et offrent une base décisionnelle exploitable par les équipes marketing pour réduire le taux de départ et optimiser le chiffre d'affaires.